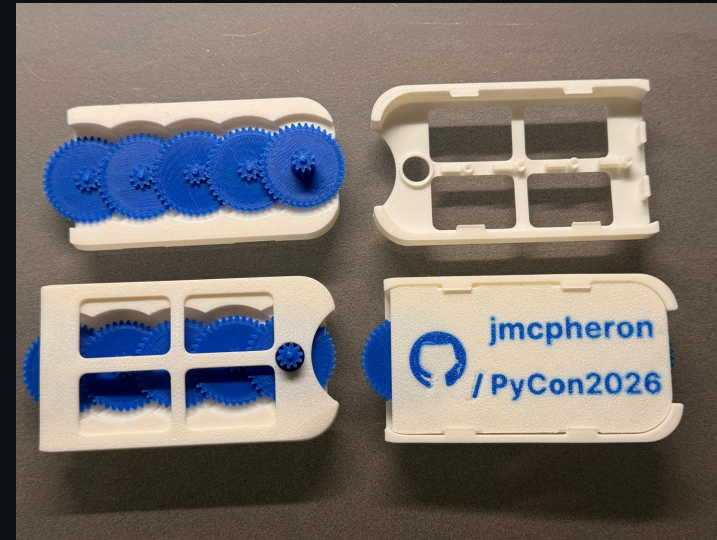


I Made a Gearbox Business Card, Then Made Python Explain It

Jason McPheron · PyCon US 2026 · Lightning Talk

Hi, I'm Jason.

- IT work, California community college — mostly student systems
- This talk is about what I do when I should be relaxing: **3D printing little mechanical things and then overthinking them with Python**
- First PyCon — wanted something physical to bring
- Something more interesting than "here's my GitHub"



So I made a 3D printed business card.

It has my GitHub project on it.

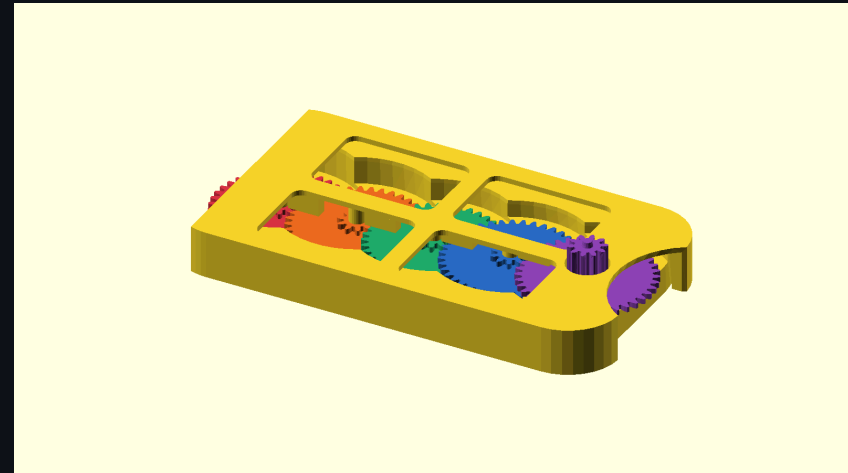
It also has a **tiny gear mechanism** built into it.

You can spin one gear with your thumb, and the motion moves through the card.

Part business card.

Part fidget toy.

Part tiny mechanical demo.



The moment the project changed.

The design started in Onshape.

Once I had the card assembled, I exported it as a **STEP file**.

```
Onshape assembly → STEP export → GitHub repo
```

That was the moment.

Because now I had **a single file** that represented the assembled physical object.

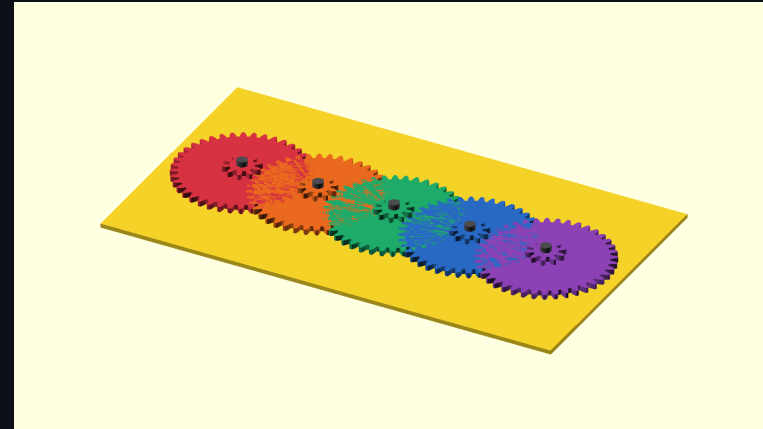
I could put it in a repo. Version it. Run tools against it. Treat the physical object a little more **like a software project**.

Python around the STEP file.

Not replacing CAD.

Tooling around the CAD export.

- Inspect the file
- Take the assembly apart
- Generate renders
- Make exploded views
- Animate the mechanism
- Keep notes, journals, tolerance experiments



A STEP file is not an explanation.

A lot of 3D printing projects end up as a pile of files.

STL. Maybe a screenshot. Maybe a README. Maybe some notes.

But it can be hard to understand what the object is, how it works, what changed, and which files matter.

So I started thinking about the repo as something that should help the object explain itself.

```
STEP file → Python tooling → renders · exploded views · animations · diagrams ·  
notes
```

GitHub Actions for physical objects.

Normally CI means tests passing, packages building, docs published.

But with a 3D object, the build output can be **visual**.

An exploded view. A render. A GIF of a tiny gearbox moving.

The repo is not just storing the design.

The repo is generating evidence about the design.

Screenshots get stale. Documentation gets stale. A repo can quietly start lying about the object it contains.

This makes that a little harder.

Paws for animals.

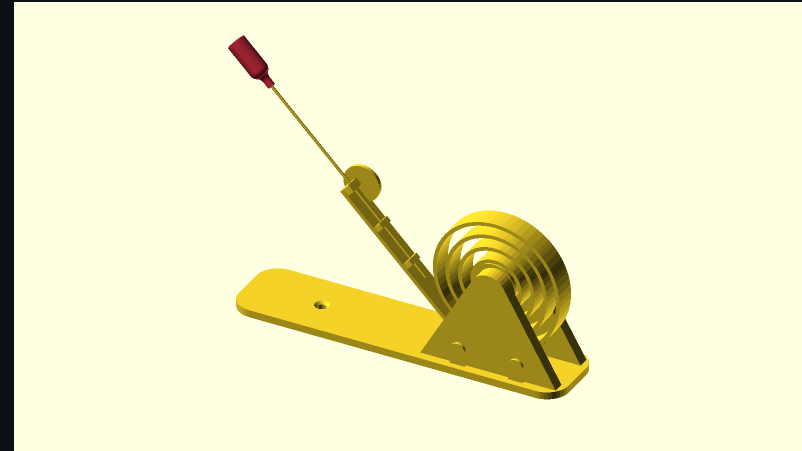
Before this card, I made a 3D printed **cat toy** for my mom.

That project is where I really started enjoying designing separate parts and assembling them.

The only problem:

I do not have a cat.

I have a golden retriever named **Bowie**.



The cat toy. Bowie remains unimpressed.

The tiny plastic maker version.

I am sure serious hardware fields have much more mature ways to do this.

Probably industry-standard workflows, best practices, blind spots I have not even learned enough to know I have.

Mine is:

- A 3D printed gearbox business card
- A public Onshape model
- A STEP export
- A Python CLI
- GitHub Actions making pictures

Small enough to be playful.

Silly enough to experiment with.

Physical open source borrows from software.

Version control · Generated documentation · Repeatable builds

Reviewable changes · Public artifacts

And maybe: **little sanity checks that help the object explain itself.**

One more thing: I'm also curious about **licensing**.

For code, we have familiar open-source licenses.

For functional 3D design files — STEP files, things you can print and use — I'm less sure what the right answer is.

Does Creative Commons make sense? Is CC BY-SA the right model? What does *sharing* mean when someone can download the geometry, modify it, print it, remix it?

Making those questions **part of the project** instead of something I figure out later in private.

Come find me.

Onshape · OpenSCAD · Python · gears · GitHub Actions

STEP files · licensing · cat toys · golden retrievers

how much tooling is too much tooling for a tiny plastic business card



github.com/jmcpheeron/pycon2026 — I have cards, ask for one.